Capítulo 10: Los entresijos internos de Git

Tutorial de Git basado en Pro Git (2.ª ed.) · Chacon y Straub · Elaborado por Ricardo · UNIAGRARIA

Introducción

Este capítulo explica cómo funciona Git por dentro: una base de datos de objetos direccionable por contenido, sobre la que se construyen los comandos de uso diario.

10.1 Fontanería y porcelana

Los comandos de porcelana (add, commit, branch, merge…) son la interfaz amigable. Los de fontanería (plumbing: hash-object, cat-file, update-index, write-tree…) son comandos de bajo nivel pensados para encadenarse en scripts. Git es, en esencia, un sistema de archivos direccionable por contenido con una interfaz de control de versiones encima.

Todo está dentro del directorio .git:

- objects/: la base de datos de objetos.
- refs/: punteros a commits (ramas, etiquetas, remotos).
- HEAD: referencia a la rama actual.
- index: el área de preparación.
- config: configuración del repositorio.
- hooks/: scripts de enganche.

10.2 Los objetos Git

Existen cuatro tipos de objetos, identificados por su hash SHA-1:

- blob: contenido de un archivo.
- tree: un directorio; lista nombres, modos y hashes de blobs u otros trees.
- commit: apunta a un tree y a sus padres, con autor, fecha y mensaje.
- tag: etiqueta anotada que apunta a un objeto.

```
git init prueba && cd prueba
echo 'contenido de prueba' | git hash-object -w --stdin
# d670460b4b4aece5915caf5c68d12f560a9fe3e4
git cat-file -p d670460b4b4aece5915caf5c68d12f560a9fe3e4
git cat-file -t d670460b4b4aece5915caf5c68d12f560a9fe3e4   # blob

git update-index --add --cacheinfo 100644 d670460b4b4aece5915caf5c68d12f560a9fe3e4
test.txt
git write-tree
echo 'primer commit' | git commit-tree <hash-del-tree>
```

10.3 Referencias Git

Las referencias son archivos de texto con un hash, ubicados en .git/refs, que dan nombres legibles a los commits.

```
git update-ref refs/heads/main <hash-commit>
```

```
cat .git/HEAD            # ref: refs/heads/main
git symbolic-ref HEAD
git tag v1.0 <hash>      # crea .git/refs/tags/v1.0
git remote add origin <url>   # refs/remotes/origin/...
```

10.4 Archivos empaquetadores (packfiles)

Inicialmente los objetos se guardan sueltos y comprimidos. Cuando hay demasiados, o al hacer push, Git los empaqueta en .pack, guardando objetos similares como diferencias (deltas) respecto a otros, con un archivo .idx de índice.

```
git gc
git verify-pack -v .git/objects/pack/pack-*.idx
```

10.5 Las especificaciones para hacer referencia a... (refspec)

Un refspec tiene el formato +<origen>:<destino> y define cómo se asignan las referencias remotas a las locales.

```
[remote "origin"]
    url = https://github.com/usuario/repo.git
    fetch = +refs/heads/*:refs/remotes/origin/*

git push origin main:refs/heads/rama-remota
git push origin :rama-a-borrar     # eliminar una rama remota
```

10.6 Protocolos de transferencia

- Protocolo tonto (dumb, HTTP): el cliente descarga archivos uno a uno; simple pero poco eficiente.
- Protocolo inteligente (smart): cliente y servidor ejecutan procesos (send-pack/receive-pack para subir; fetch-pack/upload-pack para bajar) que negocian qué objetos faltan y transfieren un solo packfile.

10.7 Mantenimiento y recuperación de datos

```
git gc                      # recompacta y limpia
git gc --auto
git fsck                    # verifica integridad y objetos huérfanos
git reflog                  # encuentra commits "perdidos"
git branch recuperada <hash> # restaurar un commit huérfano
git count-objects -v
git prune
```

Aunque se borre una rama o se haga reset --hard, los commits suelen seguir en la base de datos y accesibles por reflog durante un tiempo (por defecto unos 90 días para los alcanzables desde el reflog).

10.8 Variables de entorno

- GIT_DIR, GIT_WORK_TREE: ubicación del repositorio y del directorio de trabajo.
- GIT_AUTHOR_NAME, GIT_AUTHOR_EMAIL, GIT_AUTHOR_DATE, GIT_COMMITTER_*: datos de autoría de un commit.
- GIT_EDITOR, GIT_PAGER: editor y paginador.
- GIT_TRACE: muestra información de depuración.

```
GIT_TRACE=1 git status
GIT_AUTHOR_DATE="2026-10-01T10:00:00" git commit -m "Commit con fecha explícita"
```

Resumen

Git es una base de datos de objetos (blob, tree, commit, tag) direccionados por SHA-1, con referencias que les dan nombre y packfiles que los comprimen. Entender los comandos de fontanería ayuda a comprender y recuperar datos, ajustar refspecs y depurar el comportamiento de los comandos de porcelana.

Referencia

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. https://git-scm.com/book/es/v2 — Capítulo 10.